

應用程式開發指南：模組化史都華平台設計與模擬工具

版本：1.10 (穩定版) 日期：2025年8月30日

目標：為遊樂設備行業的技術人員，打造一款基於 Python 的視窗版桌面應用程式，用於史都華平台 (Stewart Platform) (六自由度與三自由度) 的參數化設計、性能分析、動態模擬與驗證。

[開發者註記：版本變更日誌 (v1.9 -> v1.10 穩定版)]

- **[核心模型修正]** 引入關節安裝方式選項：基於關鍵的工程實務洞見，修正了活動平台的物理模型。現在，使用者可以在 UI 介面選擇活動平台關節的安裝方式為「上置式 (法線朝上)」或「下置式 (法線朝下)」，其中「下置式」已被設為預設值，以符合絕大多數真實世界的平台結構。此修正從根本上解決了先前版本中因模型與物理現實不符而導致的角度計算錯誤與求解器收斂失敗問題。
- **[UI術語統一]** 全域術語更新：為了提升清晰度與專業性，對整個應用程式 UI 介面中的平台命名進行了統一。原有的「下平台/上平台」已被全面更新為「固定平台/活動平台」。此變更涵蓋了所有群組標題、參數標籤及 3D 視圖中的註記。
- **[UI一致性修正]** 補全三自由度診斷介面：為三自由度平台的「平台計算與分析」區塊，補上了先前版本中缺失的「零位姿態所需固定平台擺角」與「零位姿態所需活動平台擺角」兩個診斷資訊欄位，確保了兩種平台類型具備一致的功能與資訊回饋。
- **[視覺化微調]** 3D 視圖體驗優化：
 - 中心點標示：將 `Mobile Center` 更新為 `Motion Center`；縮小了中心點標記的尺寸 (`STYLE_CENTER_POINT_SIZE = 5`)；並分離了標記點與文字標籤，將文字進行偏移顯示，徹底解決了座標值被遮擋的問題。
 - 法線粗細：透過改用 `pyvista.Tube` 進行繪製，精確地將法線箭頭的粗細調整到與連桿線相匹配的視覺效果。
- **[新增]** 開發歷程記錄：在第六章中，新增了 v1.10 開發與除錯的完整記錄，詳細描述了從發現 145° 異常擺角，到最終根據工程實務修正物理模型的整個關鍵決策過程。

第一章：專案初始化與環境設定

本章節指導如何在開發電腦上建立專案的初始架構與環境。

1.1 專案目錄架構

建議在 `D:\Python_Programs\Stewart_Platform` 路徑下建立以下目錄結構，以反映 v1.7 的最新架構：

```
D:\PYTHON_PROGRAMS\STEWART_PLATFORM\  
├── .venv/  
├── data/  
│   └── projects/  
├── src/  
│   ├── core/  
│   │   ├── __init__.py  
│   │   ├── config.py  
│   │   ├── analysis_engine.py  
│   │   ├── dynamics_engine.py  
│   │   └── kinematics.py
```

```
├── state_manager.py
├── gui/
│   ├── __init__.py
│   ├── controls/
│   │   ├── __init__.py
│   │   ├── analysis_widget.py
│   │   ├── dynamics_widget.py
│   │   ├── master_geometry_widget.py
│   │   ├── six_dof_widget.py
│   │   └── three_dof_widget.py
│   ├── geometry_widget.py
│   ├── main_window.py
│   └── pyvista_widget.py
├── tests/
├── .gitignore
├── main.py
└── requirements.txt
```

1.2 虛擬環境建置

使用虛擬環境可以隔離專案依賴，避免與系統全域套件衝突。

1. **開啟終端**: 在 VS Code 中，點擊 **Terminal > New Terminal**。
2. **確認路徑**: 確保終端機的路徑在 **D:\Python_Programs\Stewart_Platform**。
3. **建立虛擬環境**: 執行以下指令，建立名為 **.venv** 的虛擬環境。

```
python -m venv .venv
```

4. **啟動虛擬環境**:

```
.\.venv\Scripts\activate
```

啟動成功後，終端機提示符前會出現 **(.venv)** 字樣。

5. **(可選) 選擇直譯器**: VS Code 可能會自動偵測到虛擬環境。如果沒有，手動點擊右下角的 **Python** 版本，選擇 **.venv** 中的 **Python** 直譯器。

1.3 前置套件安裝

1. 在專案根目錄下建立 **requirements.txt** 檔案，並填入以下內容：

```
numpy
scipy
PyQt6
pybullet
reportlab
```

```
pyvista
pyvistaqt
```

2. 在已啟動虛擬環境的終端機中，執行以下指令來安裝所有必要的套件：

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

第二章：核心理念與技術棧

2.1 核心理念

- **模組化與職責分離: (v1.7 核心升級)** 應用程式採用嚴格的四層架構，將狀態管理、核心計算、高層分析與使用者介面徹底解耦，確保了程式碼的高內聚、低耦合，便於獨立測試、維護與未來擴展。
- **引導式計算:** 程式應作為設計夥伴，能理解參數間的依賴關係，在資料不完整時引導使用者補全，並自動推算可衍生的參數。
- **設計驗證:** 內建驗證機制，主動檢查幾何矛盾與物理極限，確保設計的嚴謹性與安全性。
- **視覺化回饋:** 提供即時的 3D 視覺化回饋，讓設計過程更直觀。
- **中央化設定：** 將所有可調整的預設值、樣式和演算法參數集中到單一的設定檔 (`config.py`) 中，實現設定與程式碼的分離，便於快速微調與維護。
- **使用者狀態追蹤：** 應用程式應能追蹤使用者的修改狀態，並在關閉前主動提示儲存未保存的變更，確保工作不會意外遺失。

2.2 建議技術棧

- 程式語言: Python 3.x
- 數值/科學計算: NumPy, SciPy
- GUI 框架: PyQt6
- 3D 視覺化:
 - **PyVista / PyVistaQT:** (現行方案) 基於 VTK 的高階 API，已在本專案中用於在 PyQt 應用程式中嵌入高效能、互動式的 3D 場景，負責平台的幾何結構與姿態顯示。
- 資料庫: SQLite (用於內建組件資料庫)
- 報告生成: ReportLab / FPDF (用於匯出PDF報告)

第三章：變數與參數定義

本章定義了應用程式中使用的所有關鍵變數，作為唯一的參考標準。

- **GUI 與核心引擎單位約定:**
 - **GUI 層:** 為了符合使用者習慣，所有在圖形介面 (`GeometryWidget`, `DynamicsWidget`) 的輸入框中，長度單位皆為 **毫米 (mm)**。
 - **核心引擎層 (`CoreEngine`):** 為了符合科學計算標準，所有內部計算與參數儲存，長度單位皆為 **米 (m)**。

- **資料流:** GUI 層在將參數傳遞給核心引擎前，負責將 **mm** 轉換為 **m**。核心引擎回傳計算結果時，GUI 層負責將 **m** 轉換回 **mm** 進行顯示。
- **加速度單位:** GUI 層為了方便使用者輸入，線性加速度單位使用**重力加速度 (g)**。在將參數傳遞給核心引擎前，GUI 層負責將 **g** 值乘以 **9.81** 轉換為核心引擎所使用的 **m/s²** 單位。
- **幾何參數:**
 - R_a (m): 固定平台節點圓的半徑。
 - R_b (m): 活動平台節點圓的半徑。
 - H (m): **自然平衡零位高度 (Natural Balance Zero Position Height)**。此為一個計算結果，定義為當所有電動缸長度均處於其可用行程中點 ($L + s_{\text{buffer}}/2 + s/2$) 時，平台在保持水平姿態下的高度。它是所有姿態控制和工作空間分析的操作基準零點。
 - h (m): **平台初始高度 (Initial Height)**。此為一個理論上的物理量，定義為當所有電動缸長度均處於其**最小機械長度** (L) 時，平台在保持水平姿態下的高度。它代表了平台的**物理下限**，對應總機械行程的起點。
 - **開發者註記：** **H** 與 **h** 的區別

H 是軟體中所有相對運動的參考原點，旨在最大化和對稱化可用工作空間。而 **h** 則代表平台的物理最低點。兩者在概念上和數值上均不相同，請務必加以區分。

 - **自然零位偏移** (T_{x0}, T_{y0}) (m): 與 H 同時解算出的，在自然平衡零位下的水平面偏移量。
 - **六自由度平台:**
 - D_f (m): 固定平台相鄰節點間的較大弦長。
 - d_f (m): 固定平台相鄰節點間的較小弦長。
 - D_m (m): 活動平台相鄰節點間的較大弦長。
 - d_m (m): 活動平台相鄰節點間的較小弦長。
 - **相位角** $\Delta\theta$ (rad 或 deg): **相位角 (Phase Angle)**，亦可稱為 **交錯角 (Stagger Angle)** 或 **扭轉角 (Twist Angle)**。
 - **定義:** 在平台的初始零位姿態下，從 Z 軸俯視，活動平台節點 B_i 的方位角與固定平台對應節點 A_i 的方位角之間的夾角。
 - **性質:** 這是一個由上下平台四個弦長 (D_f, d_f, D_m, d_m) 決定的**內建幾何特性**，而非可動的姿態參數。
 - **意義:** 此參數對於**避免奇異點 (Singularity)**、優化工作空間形狀以及改善力傳遞特性至關重要。
- **電動缸參數:**
 - L (m): 電動缸的固定最小長度。
 - s (m): 電動缸的**可用行程 (Usable Stroke)** (使用者輸入)。
 - s_{buffer} (m): 電動缸兩端預留的**安全裕量 (Safety Buffer)** 總和 (使用者輸入)。
 - s_{mech} (m): 電動缸的**總機械行程 (Total Mechanical Stroke)** (由程式計算， $s_{\text{mech}} = s + s_{\text{buffer}}$)。
 - l_i (m): 第 i 根電動缸的當前長度。其安全運動範圍為 $[L + s_{\text{buffer}}/2, L + s_{\text{buffer}}/2 + s]$ 。
 - **enable_joint_limits** (bool): 一個布林旗標，用於決定是否在工作空間與姿態驗證的計算中，考慮關節的最大角度限制。
 - **base_joint_limit** (rad): 固定平台萬向節允許的最大擺動角度，相對於 Z 軸法線的夾角。
 - **platform_joint_limit** (rad): 活動平台萬向節允許的最大擺動角度，相對於平台自身法線的夾角。
 - **[新增] platform_joint_style** (str): 一個字串，用於定義活動平台關節的安裝方式。**'top'** 代表上置式 (法線朝上)，**'bottom'** 代表下置式 (法線朝下)。

- **視角參數 (View Parameters):**
 - **view_params** (dict): 一個包含攝影機所有狀態的字典集合，用於儲存和恢復 3D 視圖。
 - **cpos** (list): 一個列表，包含三個核心向量 [**position**, **focal_point**, **view_up**]，定義了攝影機的方位。
 - **parallel_scale** (float): 一個數值，定義了攝影機的縮放等級或遠近。
- **平台自由度與性能:**
 - T_x, T_y, T_z (m): 沿 X, Y, Z 軸的最大平移範圍。
 - α, β, γ (rad 或 deg): 繞 X, Y, Z 軸的最大旋轉範圍 (Roll, Pitch, Yaw)。
 - $v_{\max}, a_{\max}$ (m/s, m/s²): 平台的最大線速度與線加速度。
- **驅動系統參數:**
 - P_h (m/rev): 螺桿導程 (Pitch)。
 - N_1 : 傳動端 (馬達端) 皮帶輪的齒數。
 - N_2 : 從動端 (螺桿端) 皮帶輪的齒數。
 - i : 減速比， $i = N_2 / N_1$ 。
 - η_s : 螺桿傳動效率。
 - η_b : 皮帶輪傳動效率。
 - F_{rated} (N): 每根電動缸的額定最大軸向出力。
 - τ_{motor} (N·m): 馬達的額定扭力。
 - n_{motor} (rev/s): 馬達的額定轉速。
- **動力學參數:**
 - **platform_mass** (kg): 活動平台本身的質量。
 - **load_mass** (kg): 附加於活動平台的負載質量。
 - **load_com_x, load_com_y, load_com_z** (m): 負載質心 (Center of Mass) 相對於活動平台幾何中心的位置向量。
 - **lin_accel** (m/s²): 平台期望的線性加速度 (Linear Acceleration) 向量 [**ax**, **ay**, **az**]。
 - **ang_accel** (rad/s²): 平台期望的角加速度 (Angular Acceleration) 向量 [**ax**, **ay**, **az**]。
 - $\mathbf{I}$ (kg·m²): 平台與負載的轉動慣量張量。
 - g (m/s²): 重力加速度，預設為 9.81。

第四章：核心計算公式

本章節明確定義了將幾何參數轉換為數學模型所需的公式。

4.1 節點座標與連接定義

- **基準座標系與標準方位**
 - **原點:** (**0, 0, 0**) 固定於固定平台的幾何中心。
 - **座標軸:** 採用右手座標系。**+X** 軸水平向右，**+Y** 軸水平向前，**+Z** 軸垂直向上。
 - **標準初始方位:** 所有平台的初始方位定義，皆以此座標系為基準。固定平台的標準方位如圖所示，此為所有運動學與視覺化呈現的唯一標準。
- **平台方位示意圖**
 - **六自由度平台 (俯視圖):**
 - 平台沿 **Y** 軸鏡像對稱。
 - **A1-A6** 弦平行於 X 軸，位於 Y 軸負方向。

- 節點從右下角 **A1** 開始，逆時針編號。
- 三自由度平台 (俯視圖):
 - 平台沿 **Y** 軸鏡像對稱。
 - **A3** 節點位於 Y 軸正向頂點。
 - **A1-A5** 底邊平行於 X 軸，位於 Y 軸負方向。
- **4.1.1 自由度定義** 本節定義活動平台相對於固定平台的運動。所有旋轉均符合右手定則，但前端控制邏輯會進行調整以符合操作直覺。
 - **平移動作 (Translation):**
 - **Surge (X):** 沿 X 軸的左右運動。**+X** 為向右。
 - **Sway (Y):** 沿 Y 軸的前後運動。**+Y** 為向前。
 - **Heave (Z):** 沿 Z 軸的垂直運動。**+Z** 為上升。
 - **轉動動作 (Rotation):**
 - **Pitch (俯仰):** 繞 **X** 軸 (橫軸) 旋轉。操作定義：增加 Pitch 值，平台前端 (**Y+**) 抬起。
 - **Roll (滾轉):** 繞 **Y** 軸 (縱軸) 旋轉。操作定義：增加 Roll 值，平台右側 (**X+**) 下壓。
 - **Yaw (偏航):** 繞 **Z** 軸 (垂直軸) 旋轉。操作定義：增加 Yaw 值，從上方俯視，平台進行逆時針旋轉。
 - 三自由度平台的自由度: 包含 **Heave, Pitch, Roll**。
- **開發者註記 (程式碼同步):**
 - `kinematics.py` 中的 `_get_canonical_nodes` 函式是平台幾何的唯一真實來源。
 - 其內部的 `stagger_offset` 變數是相位角 $\Delta\theta$ (**Phase Angle**) 的直接程式碼實現，它根據弦長計算得出，並產生了上下平台之間關鍵的幾何扭轉。
 - `rotation_offset` 邏輯已被修正，以達成本章節 **4.1** 所定義的標準初始方位。

4.2 運動學與動力學公式

1. 向量定義: $\mathbf{l}_i = (\mathbf{T} + \mathbf{R})\mathbf{B}_i^0 - \mathbf{A}_i$
2. 逆向運動學 (**Inverse Kinematics, IK**): $\mathbf{l}_i = \|(\mathbf{T} + \mathbf{R})\mathbf{B}_i^0 - \mathbf{A}_i\|$
3. 正向運動學 (**Forward Kinematics, FK**): 求解 $\|\mathbf{l}_i\|^2 - l_i^{\text{given}}^2 = 0$
4. 逆動力學 (**Inverse Dynamics**):
 - **目標:** 求解各致動器 (電動缸) 所需的瞬時力 $F_{\text{actuators}}$ 。
 - **核心方程式:** $F_{\text{actuators}} = (\mathbf{J}^{-1})^T \cdot \mathbf{W}_{\text{ext}}$
 - 其中 $\mathbf{J}$ 是平台的雅可比矩陣 (**Jacobian Matrix**)， $\mathbf{W}_{\text{ext}}$ 是施加在平台的總外力/力矩向量 (**Wrench**)，它由重力與慣性力/力矩組成。
 - **開發者註記:** `dynamics_engine.py` 中的模型為簡化模型，主要考慮重力與達朗貝爾慣性力，暫未包含高速旋轉下顯著的科里奧利力與向心力。此外，目前的轉動慣量張量 ($\mathbf{I}$) 採用了一個固定的近似值 (`np.eye(3) * 0.1`)，並未根據平台和負載的實際幾何進行計算，此設計會影響高速轉動下的力矩計算精度，為未來版本的待改進項。

第五章：應用程式模組化架構

[開發者註記：本章節已根據 v1.7 的四層架構重構進行了全面重寫]

5.1 層次一：狀態管理層 (state_manager.py)

- 角色: **StateManager** 類別，是應用程式唯一、權威的數據中心 (Single Source of Truth)。
- 職責:
 1. 集中管理: 負責儲存所有需要跨模組共享或需要持久化的應用程式狀態，如專案路徑、工作空間分析結果 (可用與機械)、當前平台類型、**is_dirty** 標記等。
 2. 狀態接口: 提供統一的 **get/set** 接口供其他模組讀取或更新狀態，避免狀態數據散落在各個 UI 元件中。
 3. 生命週期: 其生命週期與主視窗相同，確保在整個應用程式運行期間，狀態都是一致且可追溯的。

5.2 層次二：核心計算層

- 1. 運動學核心 (kinematics.py)
 - 角色: **CoreEngine** 類別，專注於純粹的運動學與幾何求解。
 - 職責:
 1. 無狀態計算: 作為一個計算工具，它本身不儲存高層次的應用程式狀態 (如工作空間結果)。它接收基礎參數，執行計算，然後立即回傳結果。
 2. 運動學求解器: 封裝所有核心數學運算，包括逆向運動學、幾何求解、以及基於 **SLSQP** 最佳化的工作空間邊界分析。
 3. [修改] 姿態驗證: 提供 **is_pose_valid** 接口，用於快速檢查一個給定的姿態是否在指定的行程限制內物理可達。此函式現在能夠根據使用者選擇，額外考慮平台的萬向節角度限制，並能根據選擇的「關節安裝方式」採用正確的物理模型進行驗證。
 4. 檔案處理: 負責將專案的所有參數序列化至 JSON 檔案或從中讀取。
- 2. 動力學核心 (dynamics_engine.py)
 - 角色: **DynamicsEngine** 類別，專注於底層的逆動力學計算。
 - 職責:
 - 接收平台的瞬時幾何狀態與運動參數 (加速度)。
 - 建立平台的雅可比矩陣，並求解出各致動器所需的瞬時出力。

5.3 層次三：高層分析層 (analysis_engine.py)

- 角色: **AnalysisEngine** 類別，一個獨立的、執行緒安全的背景工作者。
- 職責:
 1. 執行複雜任務: 專門負責執行如「全域最大出力分析」這類計算密集、耗時長的分析任務，避免主 UI 執行緒被阻塞。
 2. **LHS+SQP** 演算法: 內部實作了基於拉丁超立方採樣與序列二次規劃的混合智慧演算法，用於在整個工作空間內尋找極值點。
 3. 執行緒安全: 被設計為一個無狀態的物件。它不直接存取任何主執行緒的物件，而是透過一個「任務資料包」接收工作所需的所有數據，並透過 PyQt 的信號與槽機制 (**pyqtSignal**) 安全地將進度與結果回傳給主視窗。
 4. 雙模式分析: 能夠根據從 **MainWindow** 傳入的旗標，決定是執行純靜態分析 (強制加速度為零)，還是執行包含加速度的動態分析。

5.4 層次四：使用者介面與協調層 (GUI)

- 1. 主視窗 (`main_window.py`)
 - 角色: `MainWindow` 類別，應用程式的UI 總管與事件協調中心。
 - 職責:
 1. **UI 組合**: 建立並管理所有 UI 元件的佈局與顯示。
 2. **事件協調**: 這是其核心職責。它負責接收來自 UI 元件的訊號 (如按鈕點擊)，然後決定呼叫哪個後端模組來處理該事件。
 3. **資料流控制**: 負責在各個層次之間傳遞資料。例如，從 `CoreEngine` 獲取計算結果，一份儲存到 `StateManager`，另一份轉換單位後傳遞給 `AnalysisWidget` 顯示。
 4. **執行緒管理**: 負責創建、啟動和安全地清理 `AnalysisEngine` 的背景執行緒。在啟動前，它會從 `StateManager`、`CoreEngine` 以及 `DynamicsWidget` 收集數據，打包成「任務資料包」後再派發給背景執行緒。
- 2. 各功能面板 (`controls/` 目錄下的元件)
 - 角色: 專用的 UI 元件，負責特定功能的「顯示」與「使用者輸入」。
 - `dynamics_widget.py`: (**v1.8 升級**) 其「全域分析」區塊已被修改。原按鈕更名為「分析全域最大出力」，並新增了一個「僅考慮靜態負載 (忽略加速度)」的核取方塊，讓使用者可以自由選擇分析模式。
 - **[修改]** `geometry_widget.py` / `three_dof_widget.py`: (**v1.10 升級**) 在「關節物理限制」區塊，新增了「活動平台關節安裝方式」的選項，允許使用者定義正確的物理模型。同時，所有術語已更新為「固定平台/活動平台」。
 - 職責: 每個 Widget (如 `DynamicsWidget`, `AnalysisWidget`) 都像一個儀表板。它們的職責是：接收 `MainWindow` 傳來的數據並顯示；監聽使用者的操作 (如拖動滑塊)，並將這些操作以 `pyqtSignal` 的形式發射出去，通知 `MainWindow` 進行處理。它們自身不包含複雜的應用程式邏輯。

第六章：開發者註記與決策記錄

本章旨在記錄在達成各穩定基線的過程中，所遇到的關鍵挑戰、錯誤分析以及最終的解決方案。這份記錄是未來維護與功能迭代的寶貴資產，旨在避免重蹈覆轍。

- 6.1 ~ 6.8 (前八項迭代修正記錄與 v1.9 版本相同)
- **[新增] 6.9 迭代修正六：v1.10 物理模型修正與 UI/UX 最終完善**
 - 主題：根據工程實務，從根本上修正活動平台的物理模型，並統一全域 UI 術語，解決一系列因此產生的連鎖問題，最終達成一個功能完整且互動邏輯清晰的穩定版本。
 - 遇到的问题与决策记录：
 1. **[關鍵] 根本原因診斷 (來自使用者的工程洞見)**：在 v1.9.x 的反覆測試中，即使程式邏輯不斷完善，「啟用關節限制後計算零位必定失敗」的問題依然存在。最終，由使用者提出了一個決定性的觀點：「真實世界的活動平台關節大多為下置式安裝，其法線應朝下計算」。這一洞見揭示了先前所有失敗的根源：我們的軟體一直採用一個與工程現實不符的「上置式」物理模型，導致計算出的理論擺角 (~145°) 遠超物理極限，求解器回報失敗是完全正確的。
 2. **UI 術語混淆**：使用者回饋，UI 中使用的「基座」與「平台」等術語容易造成混淆，尤其是在指代活動平台時不夠清晰。

3. 視覺化細節：使用者回饋 3D 視圖中的中心點標記過大且遮擋座標文字，法線箭頭過粗，影響了視覺體驗。
 - 最終方案：確立 **v1.10** 穩定基線：
 1. 修正核心物理模型：在 `kinematics.py` 中，新增了 `platform_joint_style` 參數。`_calculate_joint_angles` 函式現在會根據此參數，決定平台法線的方向（朝上或朝下），從而使用正確的物理模型進行角度計算。
 2. 升級 UI 功能：在 `geometry_widget.py` 中，新增了「活動平台關節安裝方式」的單選按鈕，並根據使用者的建議，將符合工程實務的「下置式 (法線朝下)」設為預設選項。
 3. 統一全域術語：對所有 `gui/` 目錄下的相關檔案進行了一次全面的文字審查與替換，將「下平台/上平台」統一更新為「固定平台/活動平台」，並將 3D 視圖中的 `Mobile Center` 更新為 `Motion Center`。
 4. 完善診斷功能：為 `three_dof_widget.py` 補上了缺失的「零位姿態所需擺角」的診斷資訊欄位，確保了 UI 的一致性。
 5. 微調視覺效果：在 `config.py` 中縮小了中心點標記的尺寸 (`STYLE_CENTER_POINT_SIZE = 5`)；在 `pyvista_widget.py` 中分離了中心點的標記與文字，並對文字進行偏移，以解決遮擋問題；同時，改用 `pyvista.Tube` 繪製法線，使其粗細與連桿匹配。
-

第七章：推薦的第三方函式庫與資源

1. 3D 視覺化: `PyVista` 與 `PyVistaQT`

- 用途: 這兩個函式庫是實現**模組 5.3** 中 3D 視覺化介面的核心。`PyVista` 提供了一套對 `VTK` 的高階封裝，簡化了 3D 物件的創建與操作；`PyVistaQT` 則提供了 `QtInteractor` 元件，讓 `PyVista` 渲染視窗可以成為獨立視窗並與主程式互動。
- 資源: `PyVista` 官方文件 (<https://docs.pyvista.org/>) 及 `PyVistaQT` GitHub 儲存庫。

2. 物理模擬 (未來擴展): `PyBullet`

- 用途: 未來若要擴展動態模擬、碰撞檢測等功能，`PyBullet` 將會是首選的物理引擎。
- 資源: `PyBullet` 官方指南 (<https://pybullet.org/wordpress/>)。